Laboratorio 4: REST APIs

HTTP, consumo desde Java, protocolos y diseño de una API con OpenAPI

Ingeniería de Software II (2016702)

Daniel Alonso Gracia Pinto

Noviembre de 2026

Grupo 00

Ejercicio 0. Anatomía de una petición HTTP

Las peticiones van a `https://httpbin.org`, que devuelve en el cuerpo lo que recibió: los parámetros, los headers y el cuerpo de la petición.

(a) Partes de la URL

La petición es `GET https://httpbin.org/get?curso=seii&grupo=3`.

Parte	Valor
Esquema	`https`
Host	`httpbin.org`
Ruta	`/get`
Parámetros de consulta	`curso=seii`, `grupo=3`

El signo `?` separa la ruta de los parámetros y `&` separa un parámetro del siguiente. En la respuesta, los parámetros aparecen en el campo `args`, ya convertidos en un objeto: `{"curso": "seii", "grupo": "3"}`. El `3` llega como texto, porque en la URL todo es texto y el servidor no sabe que era un número.

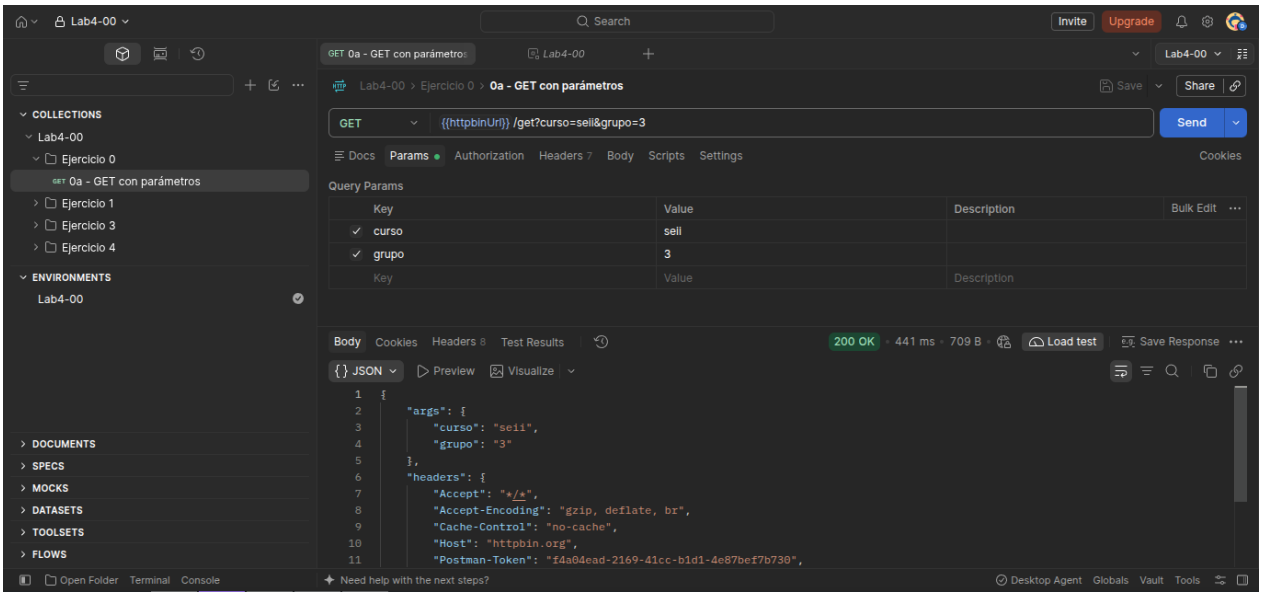


Figura 1. `GET /get` con los parámetros `curso` y `grupo`.

(b) Headers

Además del `X-Equipo: Musubi Japan`, Postman agregó por su cuenta siete headers: `Cache-Control` (`no-cache`), `Postman-Token` (un identificador distinto en cada envío), `Host`, `User-Agent` (`PostmanRuntime/2.12.2`), `Accept` (`*/*`), `Accept-Encoding` (`gzip, deflate, br`) y `Connection` (`keep-alive`). El campo `headers` de la respuesta los muestra todos menos `Connection`: ese header solo vale entre dos equipos vecinos de la red, y el intermediario que tiene httpbin.org adelante no lo reenvía.

`User-Agent`. Identifica el programa que hace la petición. El servidor lo usa para estadísticas, para adaptar la respuesta o para bloquear clientes. Con `curl` la misma petición llega con `curl/8.18.0`.

`Accept`. Dice qué formatos de respuesta acepta el cliente. `*/*` significa cualquiera; una app que solo entiende JSON debería enviar `application/json`.

`Host`. Dice a qué sitio va la petición. Un mismo servidor, con una sola IP, puede atender muchos dominios, y por el `Host` sabe cuál se pidió. En HTTP/1.1 es obligatorio.

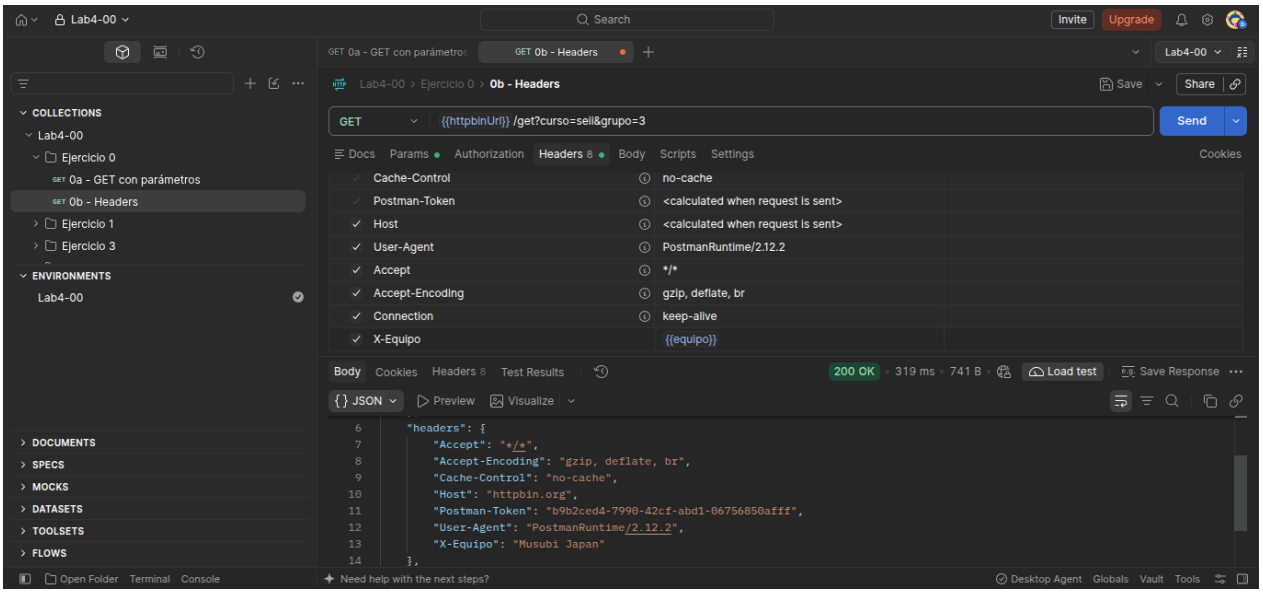


Figura 2. La misma petición con el header `X-Equipo`.

(c) Cuerpo JSON

El cuerpo enviado a `POST https://httpbin.org/post` es:

```
{
  "equipo": "Musubi Japan",
  "grupo": 0,
  "integrantes": ["Daniel Alonso Gracia Pinto"],
  "proyecto": "App móvil para aprender japonés"
}
```

El header que le dice al servidor que el cuerpo es JSON es `Content-Type: application/json`. Postman lo pone solo al elegir *raw* y *JSON*. La respuesta trae el cuerpo dos veces: en `data`, como texto, y en `json`, ya convertido en objeto.

Al cambiar a *Text*, Postman envía `Content-Type: text/plain`. El httpbin.org de hoy sigue llenando `json` con ese header: intenta leer el cuerpo como JSON sin mirar el `Content-Type`. Lo comprobé con un cuerpo que no es JSON (`hola equipo`): ahí `json` sale en `null` y el texto solo queda en `data`. El httpbin original, en cambio, devolvía `json: null` siempre que el tipo no fuera `application/json`. Eso es lo que debería hacer una API real: no adivinar el formato, sino responder `415 Unsupported Media Type` si el `Content-Type` no es el que espera.

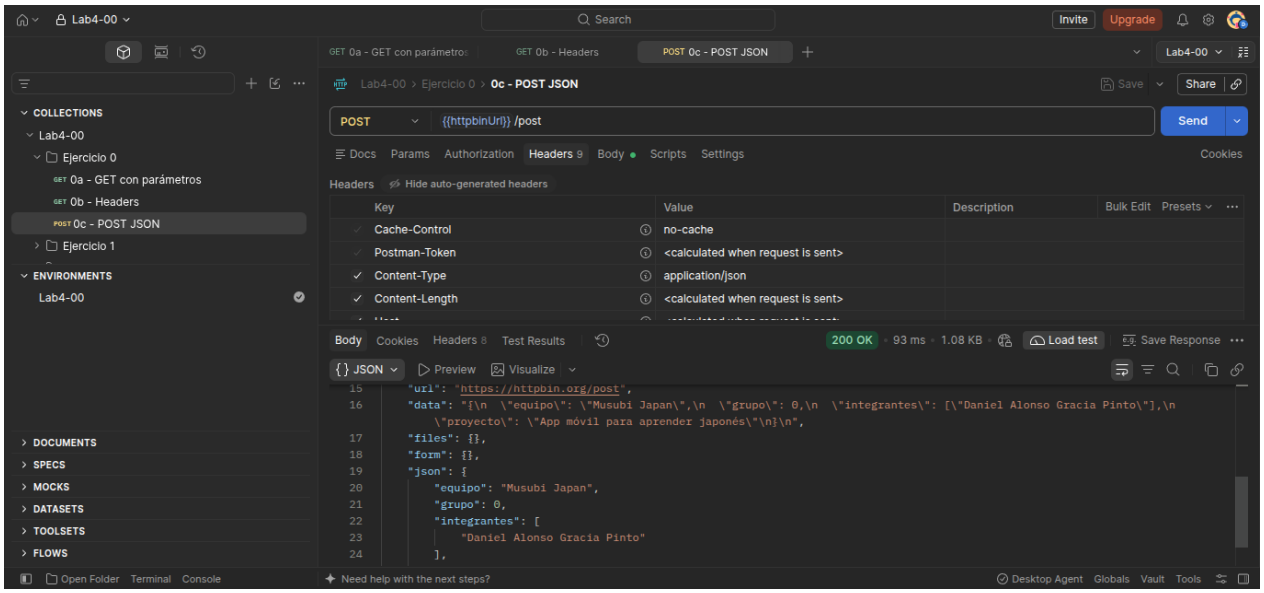


Figura 3. `POST /post` con el cuerpo como JSON.

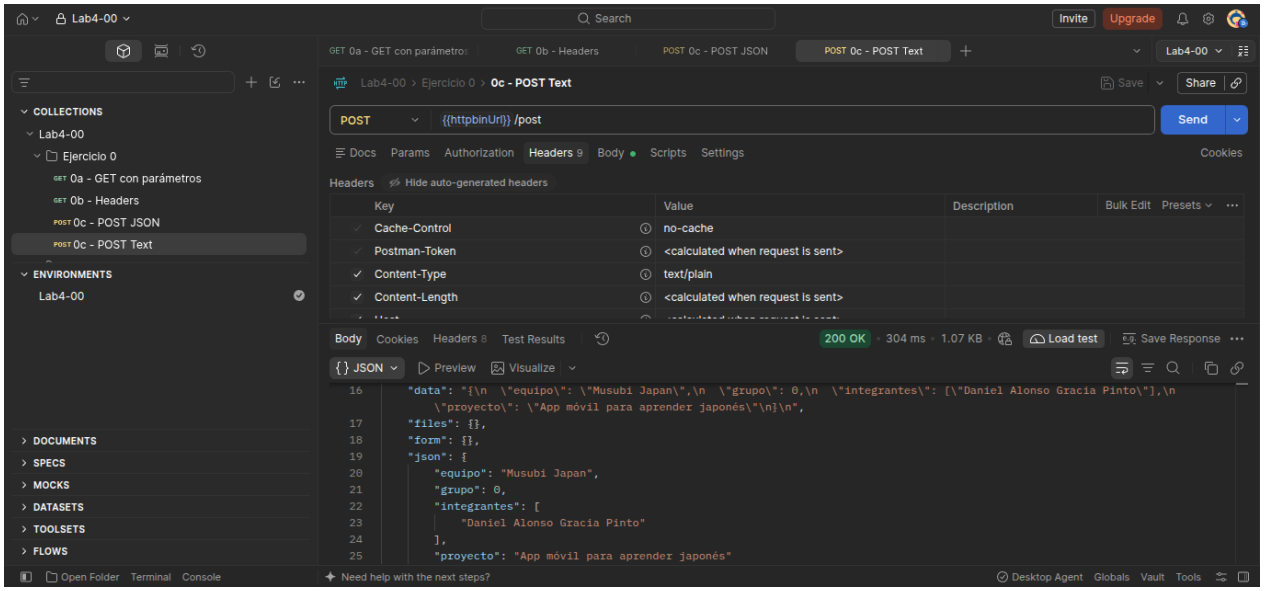


Figura 4. El mismo cuerpo enviado como *Text*: `json` sigue lleno.

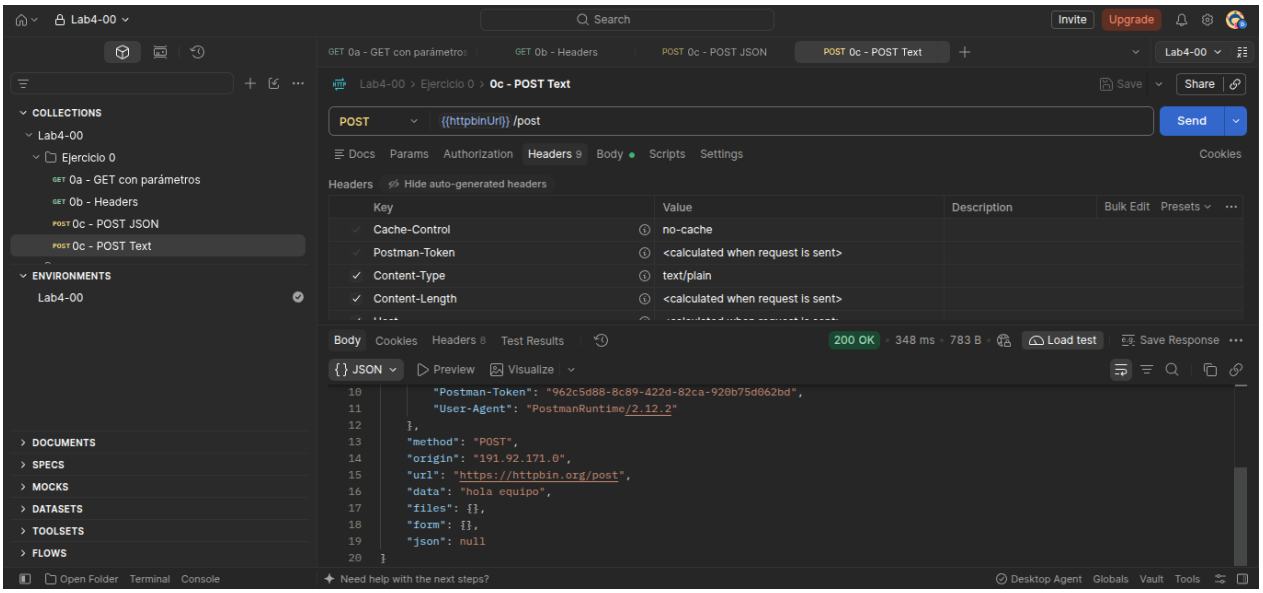


Figura 5. Un cuerpo *Text* que no es JSON: `json` sale en `null`.

(d) Códigos de estado

Las tres respuestas llegan con el cuerpo vacío (`Content-Length: 0`). La app solo cuenta con el código para decidir qué mostrar.

Código	Familia	Responsable	Qué muestra la app
201 Created	2xx	No hay error: el servidor creó el recurso.	Una confirmación (“Préstamo registrado”) y el recurso nuevo.
404 Not Found	4xx	El cliente: pidió un recurso que no existe o una URL mal armada.	“No encontramos lo que buscas” y la opción de volver. Reintentar no sirve.
500 Internal Server Error	5xx	El servidor: falló al procesar una petición válida.	“Algo salió mal de nuestro lado, intenta más tarde”, con un botón para reintentar y sin detalles técnicos.
429 Too Many Requests	4xx	El cliente: superó el límite de peticiones del servidor.	Nada de inmediato: espera lo que indique el servidor y reintenta sola. Si tarda, “Hay mucho tráfico, reintentando…”.

El 429 no se pidió: apareció al enviar la petición de (a). httpbin.org admite 17 peticiones por segundo por IP. Postman estaba usando su *Cloud Agent*, que envía la petición desde los servidores de Postman, con una IP que comparten muchos usuarios a la vez; la misma petición enviada con `curl` desde el equipo respondió 200. Al pasar al agente de escritorio, la petición salió con la IP propia y el 429 desapareció. A diferencia de los códigos de `/status`, este trae cuerpo, con el motivo y el tiempo de espera: `retry after 1 second`. Reintentar sí sirve aquí, pero esperando; reintentar enseguida solo alarga el bloqueo.

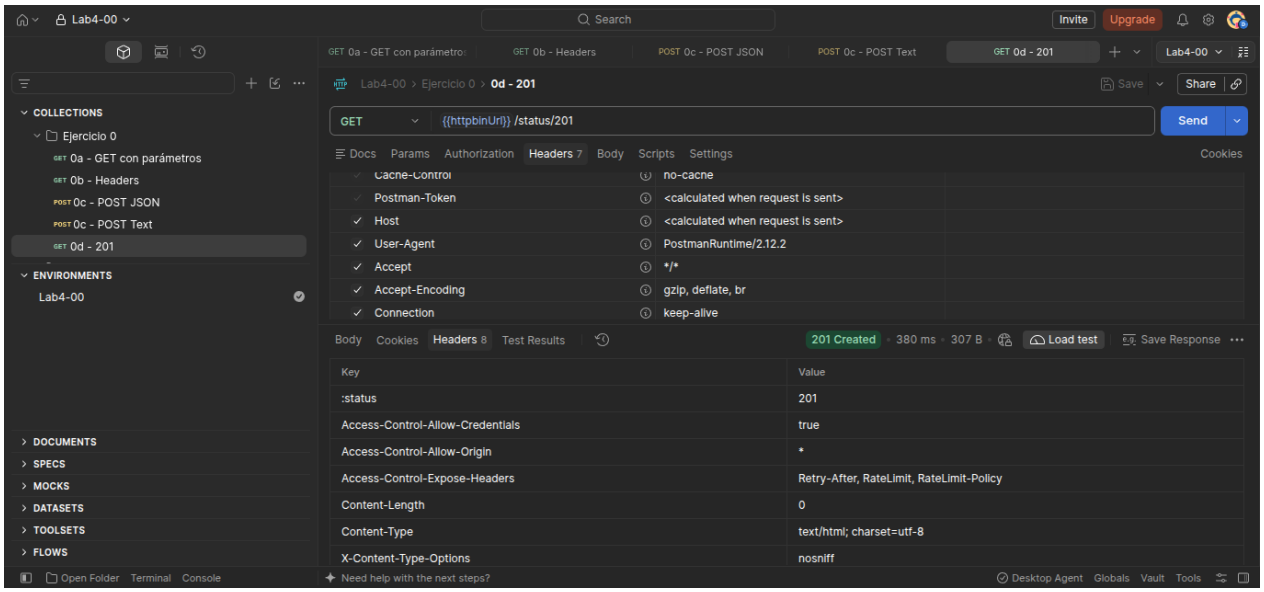


Figura 6. Respuesta de `/status/201`.

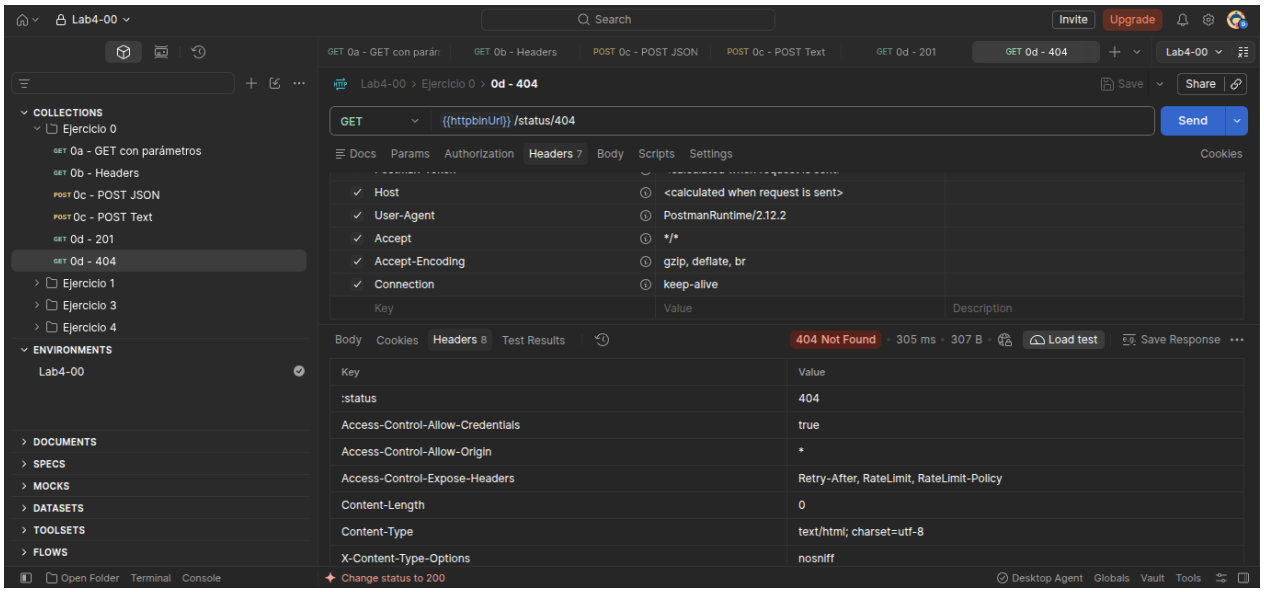


Figura 7. Respuesta de `/status/404`.

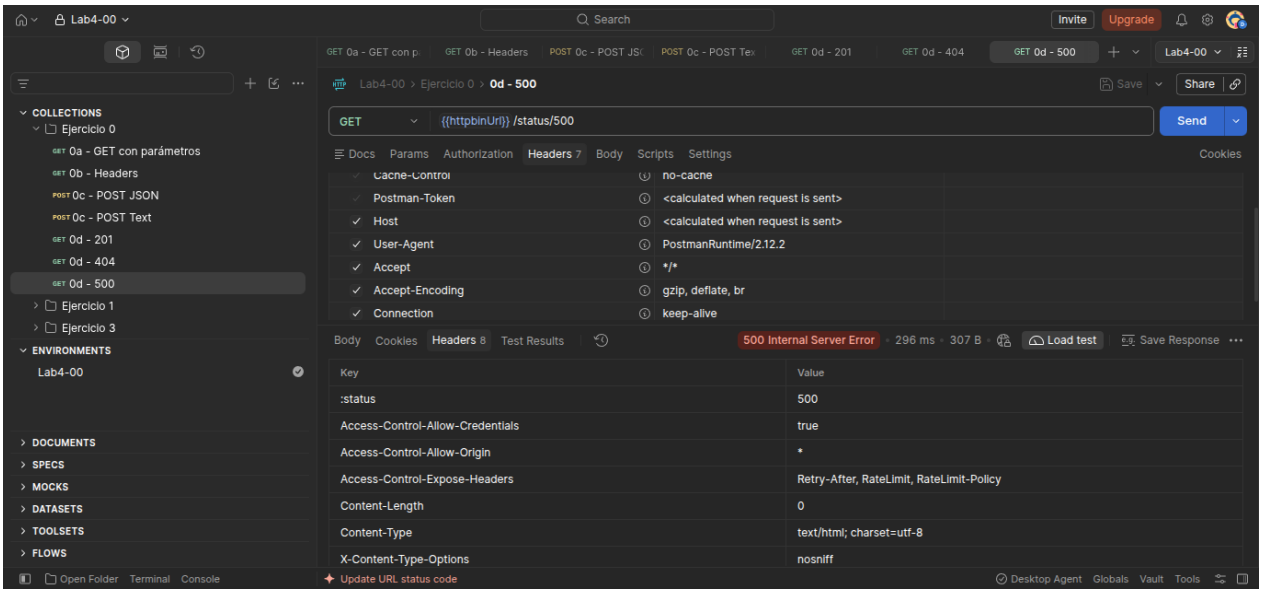


Figura 8. Respuesta de `/status/500`.

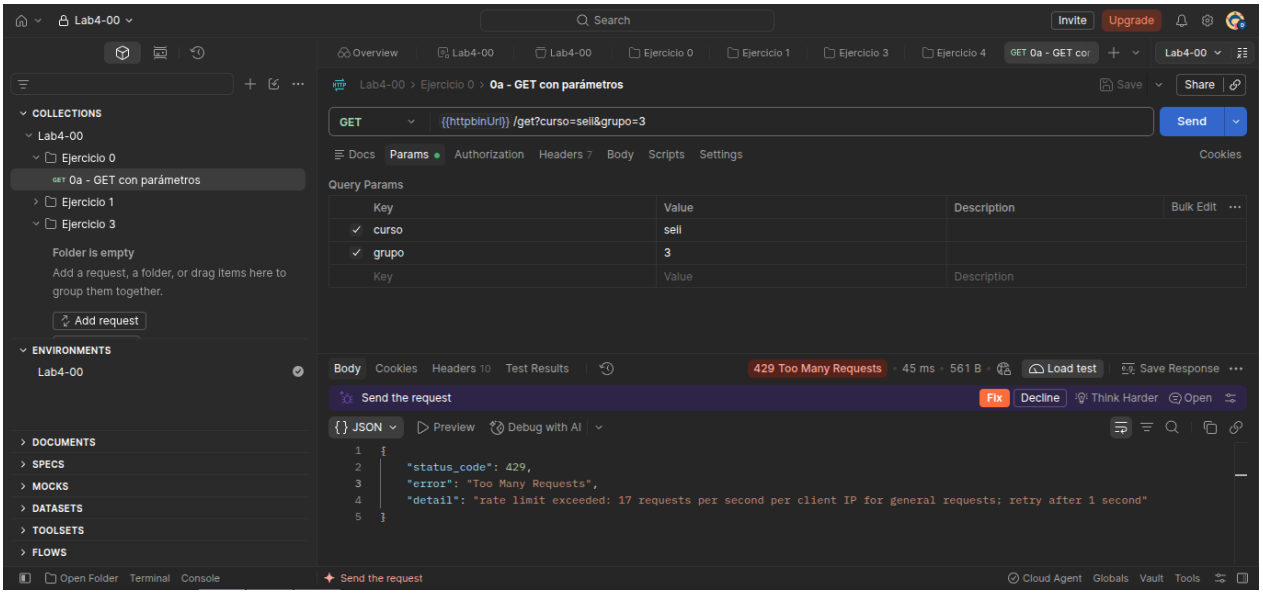


Figura 9. El 429 que devolvió httpbin.org al superar el límite de peticiones.

Ejercicio 1. CRUD completo con JSONPlaceholder

El environment define `baseUrl = https://jsonplaceholder.typicode.com`, `postId = 1` y `userId = 1`. Las peticiones usan esas variables en las rutas y en los cuerpos; la tabla muestra las URLs ya resueltas. JSONPlaceholder acepta escrituras, pero no las guarda: responde lo que respondería un servidor real.

(a) Métodos, códigos y propiedades

Un método es *seguro* si no cambia nada en el servidor, e *idempotente* si enviarlo una vez o varias deja el servidor en el mismo estado. Todo método seguro es idempotente; al revés no.

Método	Endpoint	Código	¿Seguro?	¿Idempotente?	Qué hace
GET	`/users`	200	Sí	Sí	Lista los 10 usuarios.
GET	`/users/1/posts`	200	Sí	Sí	Lista los 10 posts del usuario 1.
GET	`/posts?userId=1`	200	Sí	Sí	Filtra los posts por `userId`; devuelve los mismos 10.
POST	`/posts`	201	No	No	Crea un post. Responde con `id: 101` y `Location: /posts/101`.
PUT	`/posts/1`	200	No	Sí	Reemplaza el post 1 entero por el cuerpo enviado.
PATCH	`/posts/1`	200	No	No*	Cambia solo los campos enviados y conserva los demás.
DELETE	`/posts/1`	200	No	Sí	Borra el post 1. Responde `{}`.
GET	`/posts/9999`	404	Sí	Sí	Busca un post que no existe. Responde `{}`.

*HTTP no garantiza que `PATCH` sea idempotente. Lo es cuando fija valores, como aquí (`title = "Solo cambia el titulo"`), y deja de serlo con operaciones relativas, como "sumar 1 a las vistas".

`DELETE` es idempotente aunque la segunda vez responda distinto (en una API real, 404): lo que cuenta es el estado del servidor, y después de uno o de diez `DELETE` el post sigue sin existir. Una API real respondería `204 No Content` en lugar de `200` con un cuerpo vacío.

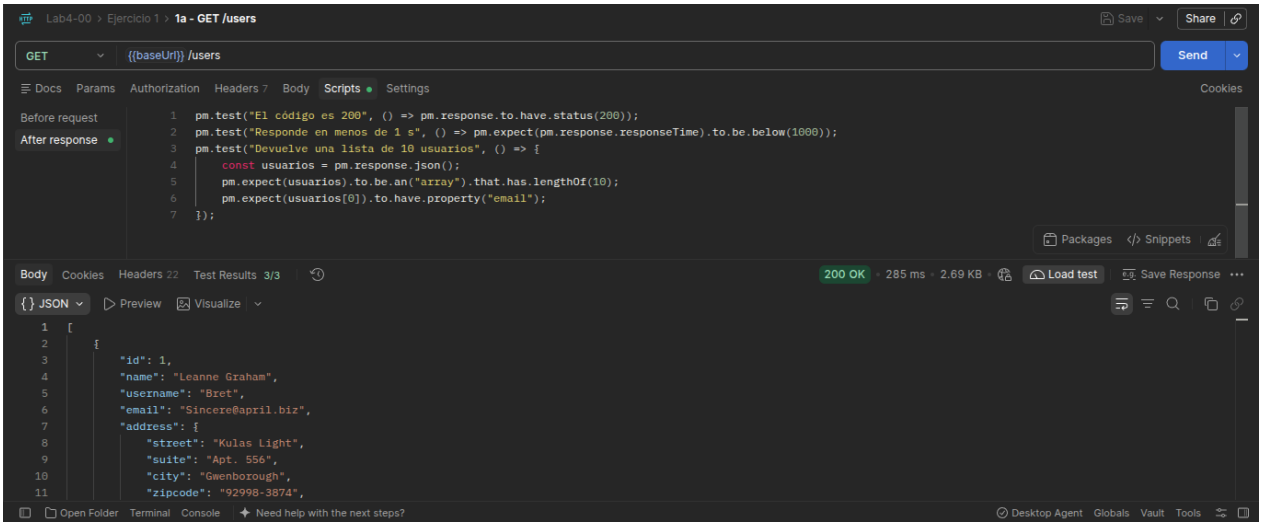


Figura 10. `GET /users`.

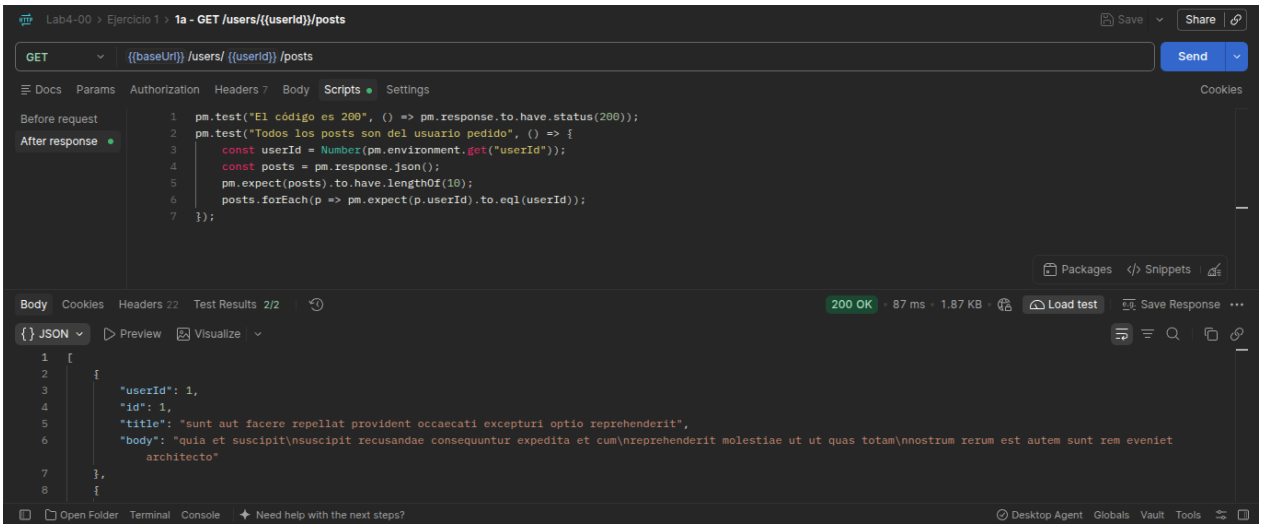


Figura 11. `GET /users/1/posts`.

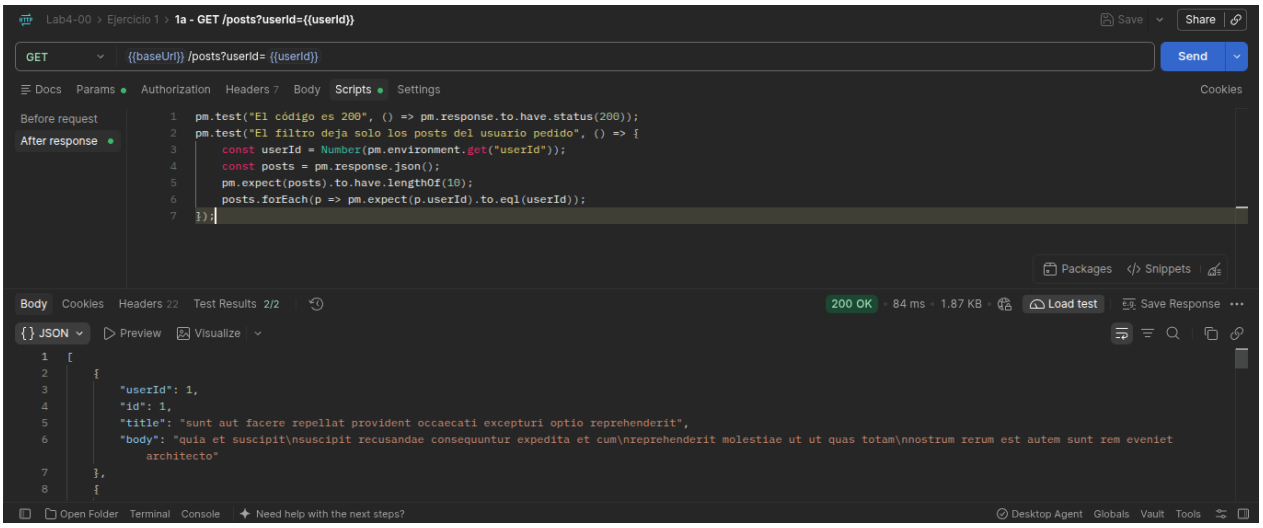


Figura 12. `GET /posts?userId=1`.

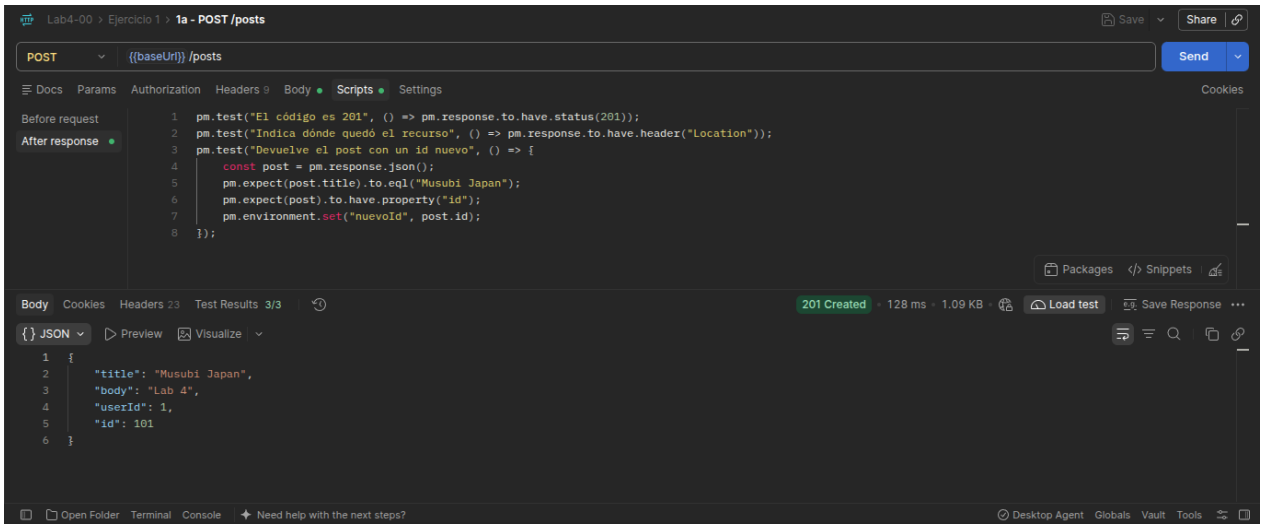


Figura 13. `POST /posts`: el cuerpo de la respuesta.

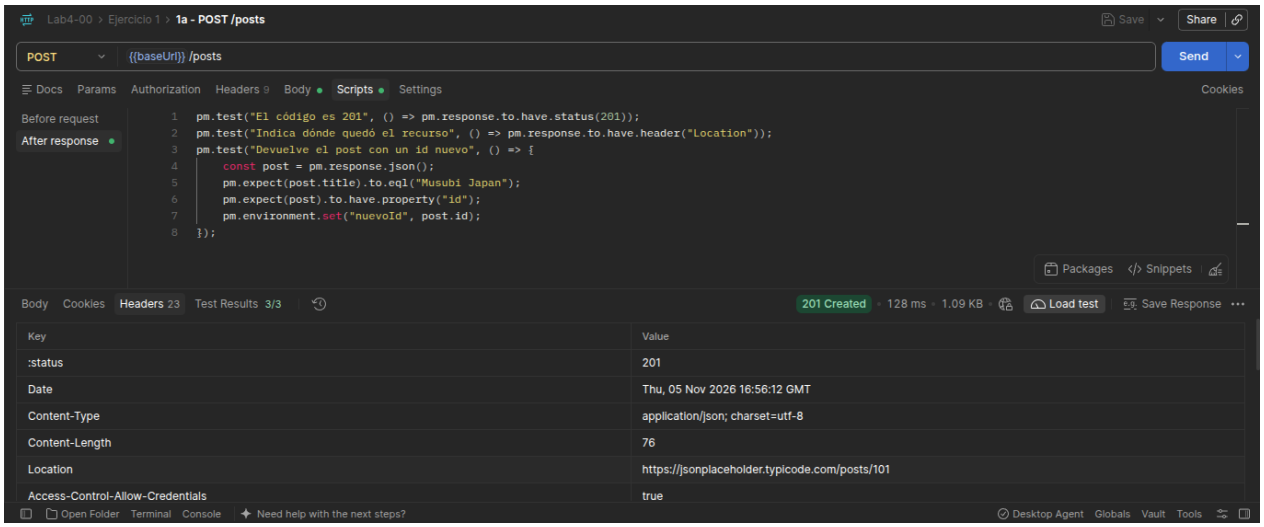


Figura 14. `POST /posts`: el header `Location` de la respuesta.

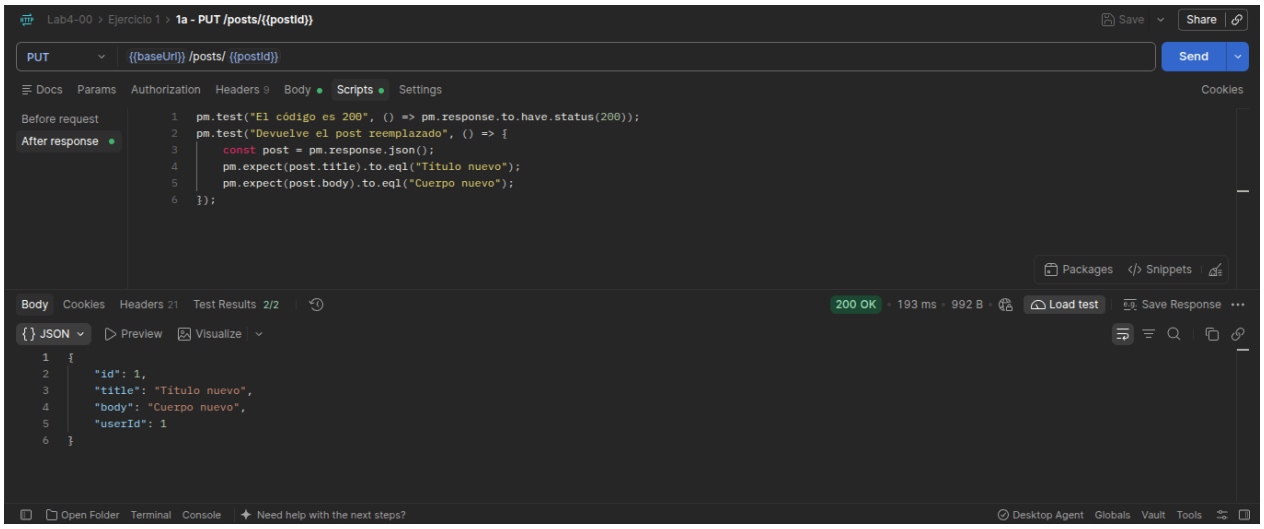


Figura 15. `PUT /posts/1`.

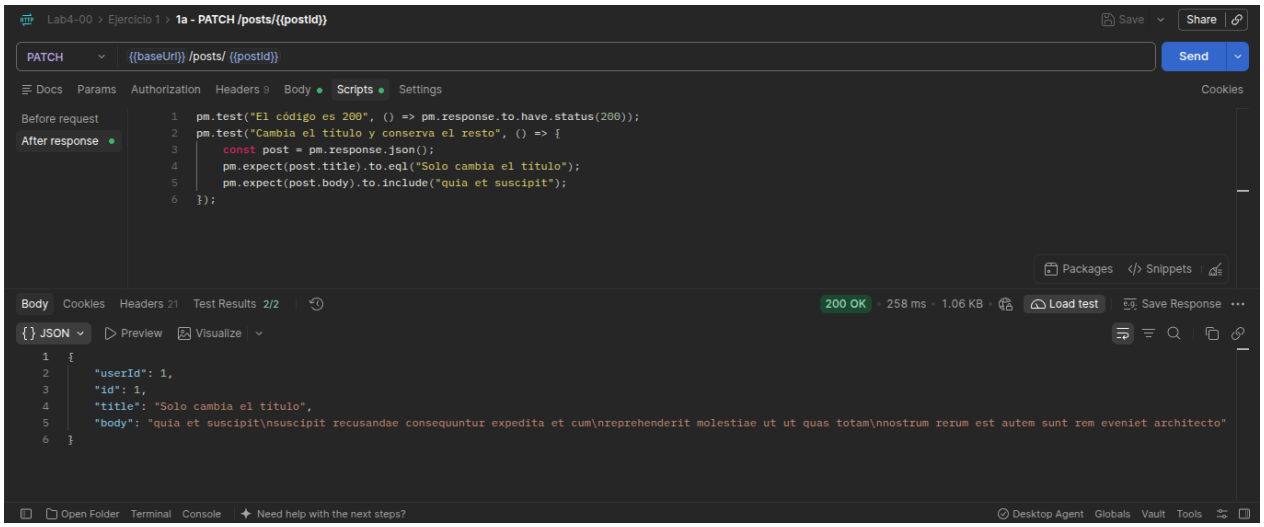


Figura 16. `PATCH /posts/1`.

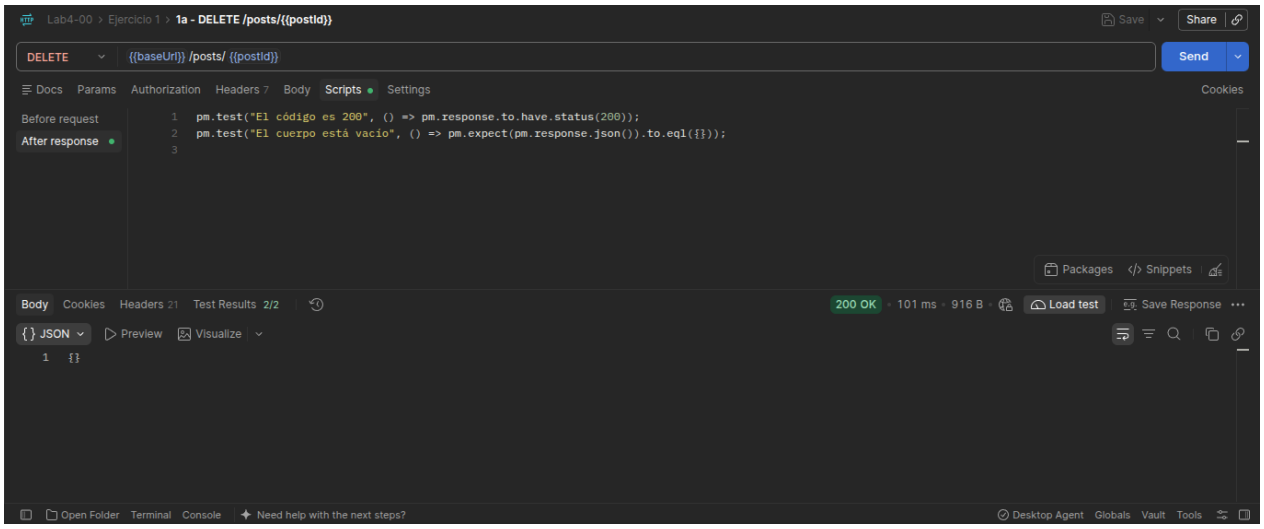


Figura 17. `DELETE /posts/1`.

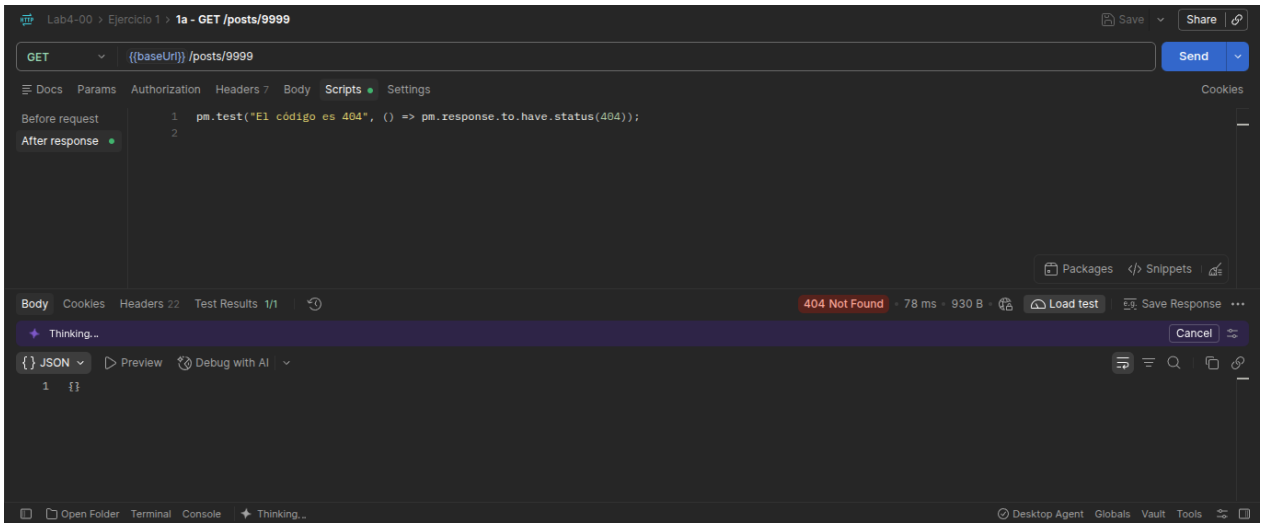


Figura 18. `GET /posts/9999`.

(b) Ruta anidada o parámetro de consulta

Las dos peticiones devuelven los mismos 10 posts, pero dicen cosas distintas. La ruta `/users/1/posts` dice que los posts *pertenecen* al usuario 1: es una relación de pertenencia. El parámetro `/posts?userId=1` dice que de la colección de todos los posts se quieren solo algunos: es un filtro.

El criterio general es que la ruta identifica un recurso y la consulta filtra una colección. Conviene la ruta anidada cuando el hijo no tiene sentido sin el padre y casi siempre se consulta desde él, como las líneas de un pedido (`/pedidos/7/lineas`); conviene un nivel de anidamiento, dos como máximo. Conviene el parámetro cuando el criterio es opcional o se combina con otros: `/posts?userId=1&q=lab&page=2` se arma sin crear rutas nuevas, mientras que con rutas habría que inventar una por cada combinación.

(c) Crear y luego consultar

El `POST /posts` respondió `201 Created`, un cuerpo con `id: 101` y el header `Location: https://jsonplaceholder.typicode.com/posts/101`. El `GET /posts/101` siguiente respondió `404 Not Found`.

JSONPlaceholder solo tiene 100 posts y no guarda las escrituras: calcula el `id` que tendría el post y arma la respuesta que daría un servidor real, pero no lo almacena. Por eso la URL del `Location` lleva a un recurso que no existe. Una prueba guarda el `id` en la variable `nuevoId`, y la petición siguiente consulta `{{baseUrl}}/posts/{{nuevoId}}`, sin escribir el 101 a mano.

En una API real, el `POST` debe responder `201 Created` con el header `Location` apuntando al recurso nuevo, y el `GET` a esa URL debe devolverlo con `200`. JSONPlaceholder imita bien la respuesta; lo que falta es el almacenamiento.

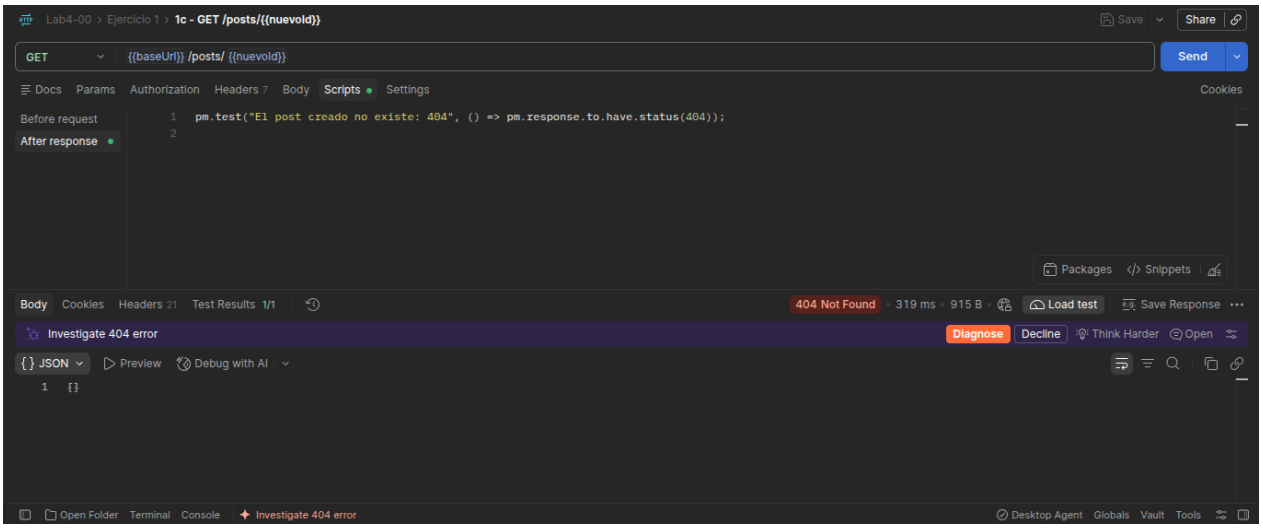


Figura 19. `GET /posts/{{nuevoId}}` después del `POST`: 404.

(d) PUT y PATCH

El `PUT /posts/1` envía el recurso completo:

```
{ "id": 1, "title": "Titulo nuevo", "body": "Cuerpo nuevo", "userId": 1 }
```

El `PATCH /posts/1` envía solo lo que cambia:

```
{ "title": "Solo cambia el titulo" }
```

La respuesta del `PATCH` trae el título nuevo y el `body` original del post 1 (`"quia et suscipit…"`), porque los campos que no se enviaron se conservan.

`PUT` significa "deja el recurso exactamente así". Si se omite `title`, el post queda sin título: JSONPlaceholder lo muestra, porque su respuesta al `PUT` sin `title` trae solo `id`, `body` y `userId`. En una API real pasaría una de dos cosas: si `title` es obligatorio, el servidor rechaza la petición con `400 Bad Request`; si es opcional, lo borra o lo deja en `null`. En ningún caso conserva el título anterior: para cambiar un solo campo se usa `PATCH`.

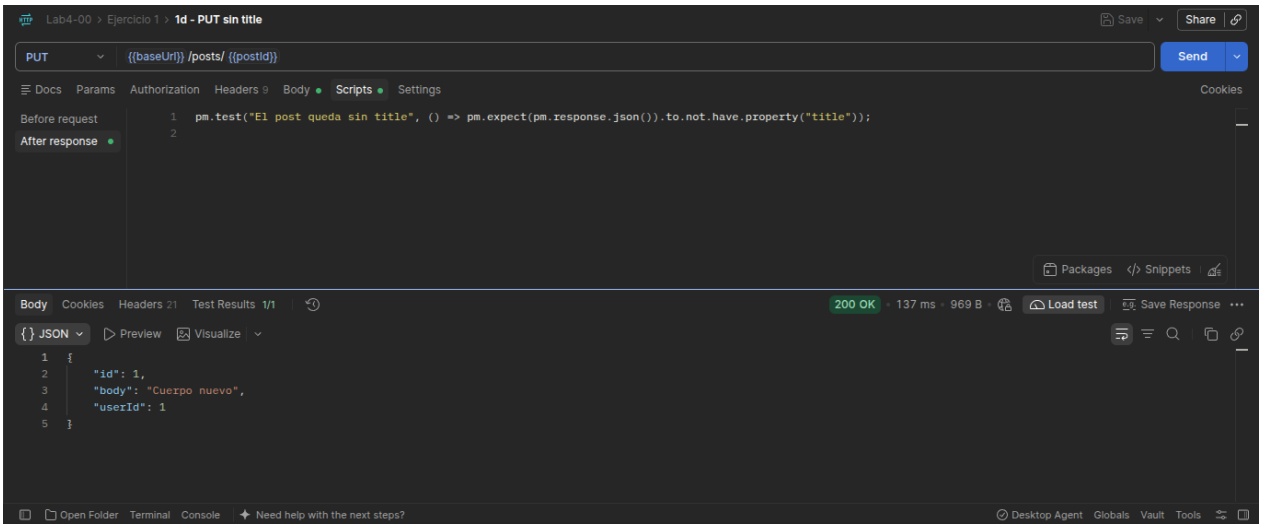


Figura 20. `PUT /posts/1` sin `title`: la respuesta no lo trae.

(e) Pruebas automáticas

Las diez peticiones de la carpeta `Ejercicio 1` tienen pruebas en *Scripts → Post-response*. Todas verifican el código de estado; además, cada una revisa lo propio de su método: que los filtros solo traigan posts del usuario 1, que el `POST` incluya `Location`, que el `PATCH` conserve el `body` y que el `PUT` sin `title` lo pierda. El archivo completo es `postman/scripts-ejercicio1.js`. Las de `GET /users` son estas:

```
pm.test("El código es 200", () => pm.response.to.have.status(200));
pm.test("Responde en menos de 1 s", () => pm.expect(pm.response.responseTime).to.be.
    below(1000));
pm.test("Devuelve una lista de 10 usuarios", () => {
    const usuarios = pm.response.json();
    pm.expect(usuarios).to.be.an("array").that.has.lengthOf(10);
    pm.expect(usuarios[0]).to.have.property("email");
});
```

El Collection Runner ejecutó la carpeta completa, en orden, con el environment `Lab4-00`: las 19 pruebas pasaron, sin errores, en 2,8 segundos. El orden importa: el `POST` va antes del `GET /posts/{{nuevoId}}` porque es el que define `nuevoId`.

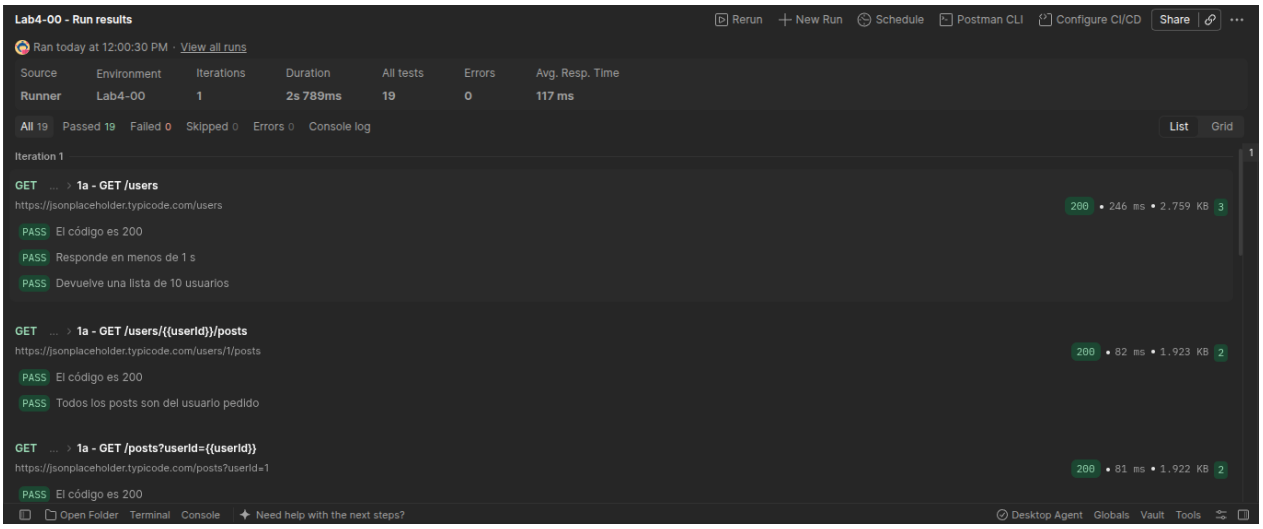


Figura 21. Resultado del Collection Runner sobre la carpeta `Ejercicio 1`.

Ejercicio 2. Un cliente REST en Java

Ejercicio 3. Los seis protocolos en la práctica

Ejercicio 4. Diseño y documentación de una API REST